# SIMATS ENGINEERING

## Descriptive Written Exam

**COURSE NAME: Software Testing** **COURSE CODE: CSA3712**

**COURSE OUTCOME COVERED**

**CO1:** *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

| Sl. No | Questions | Bloom's Level |
|---|---|---|
| 1 | A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example. | 3 |
| 2 | An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes. | 3 |
| 3 | A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples. | 3 |
| 4 | A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example. | 3 |

| | | |
|---|---|---|
| 5 | A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements. | 3 |
| 6 | A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures. | 3 |
| 7 | A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively. | 3 |
| 8 | An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage. | 3 |
| 9 | A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy. | 3 |
| 10 | A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness. | 3 |

| | | |
|---|---|---|
| 11 | A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting. | 3 |
| 12 | A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects. | 3 |
| 13 | A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures. | 3 |
| 14 | A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices. | 3 |
| 15 | A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation. | 3 |
| 16 | A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards. | 3 |
| 17 | A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency. | 3 |

| | | |
|---|---|---|
| 18 | A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design. | 3 |
| 19 | A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices. | 3 |
| 20 | A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection. | 3 |

**1. Testing as an Engineering Activity in Banking Application**

Testing should be treated as a **systematic engineering activity**, not random or ad-hoc checking. For a banking application, the testing process can be redesigned as follows:

1. **Requirement analysis:** Identify requirements such as login, balance enquiry, fund transfer, withdrawal and transaction security.
2. **Systematic test design:** Prepare test cases using techniques such as:
    - Equivalence Partitioning
    - Boundary Value Analysis
    - Decision Table Testing
    - State Transition Testing
3. **Use of testing tools:** Use test-management, automation and defect-tracking tools to execute and record test cases.
4. **Standards and procedures:** Follow defined test plans, coding standards, review procedures and documentation standards.
5. **Automation:** Automate repetitive regression tests for important banking functions.
6. **Defect management:** Record each defect with severity, priority, steps to reproduce and expected/actual results.

**Example:** For a fund transfer of ₹1,000, test valid transfer, insufficient balance, invalid account number, zero amount, negative amount and transaction-limit boundaries.

**Conclusion:** Replacing ad-hoc testing with structured test design, automation and standards provides better coverage, detects defects earlier and improves banking application reliability. This directly addresses the problem described in Question 1. CSA37-ST_CO1_AT1_Descriptive Wr…

**2. Structured Testing Process for ATM Withdrawal**

A structured testing process should be applied throughout the SDLC:

**1. Requirement Phase**

Identify requirements such as card validation, PIN verification, balance checking and cash withdrawal.

**2. Test Planning**

Define scope, test environment, resources, test data and testing schedule.

**3. Test Design**

Prepare test cases for valid and invalid withdrawals.

**4. Test Execution**

Execute test cases and compare actual results with expected results.

**5. Defect Reporting**

Record failures in a defect repository and assign severity and priority.

**6. Retesting and Regression Testing**

After fixing defects, retest the failed cases and perform regression testing.

**ATM example:**

| Test Case | Expected Result |
|---|---|
| Valid card + valid PIN + sufficient balance | Cash dispensed |
| Wrong PIN | Transaction rejected |
| Insufficient balance | Withdrawal rejected |
| Amount above daily limit | Transaction rejected |
| Invalid amount | Appropriate error displayed |

A consistent process ensures that the same important scenarios are tested every time.

**Conclusion:** A defined SDLC testing process reduces missed defects, improves consistency and produces a reliable ATM system. CSA37-ST_CO1_AT1_Descriptive Wr…

**3. Error, Defect and Failure**

These three terms describe different stages of a software problem.

**Error**

An **error** is a human mistake made during requirements, design or coding.

**Example:** A developer writes balance - withdrawal incorrectly as balance + withdrawal.

**Defect**

A **defect** is the incorrect code, design or requirement resulting from the error.

**Example:** The withdrawal module contains incorrect balance-calculation logic.

**Failure**

A **failure** occurs when the software executes the defective code and produces an incorrect result.

**Example:** A customer with ₹10,000 withdraws ₹2,000, but the application displays ₹12,000.

**Relationship**

**Human Error → Defect → Execution → Failure**

Improper understanding can cause testers to report a failure without identifying the underlying defect, making communication difficult.

**Better defect reporting**

A report should contain:

- Defect ID
- Module
- Steps to reproduce

- Expected result
- Actual result
- Severity
- Priority
- Environment
- Screenshots/logs

This makes defect tracking clearer and helps developers reproduce and fix problems efficiently. CSA37-ST_CO1_AT1_Descriptive Wr…

**4. Defect Clustering and Test Prioritization**

**Defect clustering** means that a small number of modules often contain a large percentage of defects. Therefore, testing should not distribute effort equally across every module.

**Approach**

1. Analyze historical defect data.
2. Identify modules with high defect density.
3. Identify critical business functions.
4. Give high-risk modules more test coverage.
5. Prioritize critical test cases.
6. Perform regression testing after fixes.

**Test priority**

For a banking application:

**High priority:**

- Login
- Fund transfer
- Payment
- Account balance

**Medium priority:**

- Profile update
- Notifications

**Low priority:**

- UI themes and cosmetic features

Other principles such as **exhaustive testing is impossible**, **testing shows presence of defects**, and **early testing** also help allocate testing resources.

**Example:** If 60% of previous defects occurred in fund transfer and payment modules, these modules should receive more detailed testing.

**5. Defect Classification and Defect Repository**

Defects originate from different stages such as:

- Incorrect requirements
- Design mistakes
- Coding errors
- Interface problems
- Configuration problems
- Data errors
- Integration problems

They can be categorized based on:

**Severity**

- Critical
- Major
- Minor
- Trivial

**Type**

- Functional
- Performance
- Security
- UI
- Compatibility
- Data

A **defect repository** stores historical defect information.

It should contain:

- Defect ID
- Description
- Module
- Root cause
- Severity
- Priority
- Status
- Resolution
- Release information

**Using repository data**

If previous releases show repeated login defects, testers can create additional login test cases and regression tests.

**Prevention:** Perform root-cause analysis and update test cases whenever a recurring defect is discovered.

**Conclusion:** Effective use of defect classification and repository information helps identify patterns, prevent repeated defects and improve overall software quality. CSA37-ST_CO1_AT1_Descriptive Wr…

**6. Mobile Banking Fund Transfer Crashes**

To identify the root cause of crashes:

1. Reproduce the crash consistently.
2. Check application logs and error messages.
3. Test different account types and transaction amounts.
4. Test different network conditions.
5. Test boundary values.
6. Perform integration testing between banking, database and payment services.
7. Perform stress and load testing.

**Example**

Test:

- ₹100 transfer
- ₹0 transfer
- Negative amount
- Amount exceeding balance
- Maximum transaction limit
- Slow network
- Network disconnection during transfer
- Duplicate transaction

The defect could originate from incorrect input validation, database failure, API communication or exception handling.

**Improvement:** Introduce automated regression tests, negative testing, boundary testing, integration testing and continuous defect tracking.

**Conclusion:** Systematic testing helps identify root causes instead of merely observing crashes and reduces the possibility of repeated failures. CSA37-ST_CO1_AT1_Descriptive Wr…

**7. Testing vs Debugging**

| Testing | Debugging |
|---|---|
| Finds defects/failures | Finds and fixes the cause |
| Usually performed by testers and developers | Mainly performed by developers |
| Compares expected and actual results | Examines code, logic and execution |
| Can be planned and systematic | Usually starts after a failure/defect is identified |

**Example**

Testing discovers that a fund transfer of ₹5,000 produces an incorrect balance.

The tester reports the defect.

The developer debugs the code, identifies an incorrect calculation and fixes it.

After the fix, the tester performs **retesting** and **regression testing**.

**Conclusion:** Testing and debugging are complementary. Testing identifies problems, while debugging determines and removes their causes. Confusing them can result in poor defect management. CSA37-ST_CO1_AT1_Descriptive Wr…

**8. Cross-Browser Testing**

An e-commerce application should be tested across different:

- Browsers
- Browser versions
- Operating systems
- Screen sizes
- Devices

**Strategy**

1. Identify browsers used by customers.
2. Prepare a browser/device compatibility matrix.
3. Test important functions such as login, search, cart and payment.
4. Perform UI and functional testing.
5. Perform responsive testing.
6. Automate repeated browser tests.

**Tools**

Tools such as **Selenium** and cloud-based browser testing platforms can help execute tests across multiple browser environments.

**Example**

A checkout button may work correctly in Chrome but fail to respond in another browser because of JavaScript or CSS compatibility issues.

**Conclusion:** Systematic cross-browser testing identifies compatibility defects early and improves application reliability and user experience. CSA37-ST_CO1_AT1_Descriptive Wr…

**9. Payroll Logical Errors and Defect Classification**

Incorrect salary calculation is a **functional/logic defect**.

For example:

Net Salary = Basic Salary + Allowances – Deductions

If the program incorrectly subtracts allowances, it produces an incorrect salary.

**Classification**

The defect can be categorized as:

- Functional defect
- Calculation/logic defect
- Potentially high severity because payroll affects employees financially

**Testing techniques**

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- Calculation verification
- Regression testing

**Prevention**

- Code reviews
- Unit testing
- Automated calculation tests
- Requirement reviews
- Regression testing
- Independent test data

**Conclusion:** Defect classification helps testers select appropriate testing techniques and prevents similar calculation errors in future releases. CSA37-ST_CO1_AT1_Descriptive Wr…

**10. Structured Test Case Design**

Random testing cannot guarantee adequate coverage. Testing should be redesigned using systematic test-case design.

**Steps**

1. Analyze requirements.
2. Identify test conditions.
3. Design positive and negative test cases.
4. Apply test design techniques.
5. Define expected results.
6. Review test cases.
7. Execute and document results.

**Suitable techniques**

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use-case testing
- Error guessing

**Example:** For a password field requiring 8–16 characters:

- 7 characters → Invalid
- 8 characters → Valid
- 16 characters → Valid
- 17 characters → Invalid

This provides better boundary coverage than random inputs.

**Conclusion:** Structured test-case design improves coverage, repeatability and defect detection. CSA37-ST_CO1_AT1_Descriptive Wr…

**11. Data Inconsistency in Healthcare Application**

Data inconsistency may occur because of:

- Incorrect database updates
- Interface defects
- Incorrect data mapping
- Synchronization problems
- Integration defects

**Testing approach**

1. Verify requirements and data rules.
2. Test each module independently.
3. Perform integration testing.
4. Verify database values.
5. Test concurrent updates.
6. Check data validation.
7. Perform end-to-end testing.

**Example:** If a patient's address is updated in the registration module, the same correct address should appear in billing and appointment modules.

**Defect tracking**

Every inconsistency should be recorded with:

- Module
- Input data
- Expected data
- Actual data
- Database details
- Steps to reproduce
- Severity

**Conclusion:** Systematic integration and data validation testing ensures data integrity across healthcare modules. CSA37-ST_CO1_AT1_Descriptive Wr…

**12. Defect Repository for Repeated Defects**

A defect repository provides historical information about defects.

**How it supports testing**

Testers can analyze:

- Frequently defective modules
- Common defect types
- Repeated root causes
- Defects introduced in particular releases
- Defects that frequently escape testing

**Example**

Suppose every release contains defects in the payment module.

Repository analysis may show that most defects are caused by incorrect validation.

The testing team can then create additional validation and boundary test cases.

**Improvements**

- Maintain complete defect records.
- Perform root-cause analysis.
- Convert important defects into regression test cases.
- Review defect trends after every release.
- Conduct defect-prevention meetings.

**Conclusion:** Historical defect data transforms previous failures into useful information for future test design and prevention. CSA37-ST_CO1_AT1_Descriptive Wr…

**13. Web Learning System Fails During Peak Usage**

The failure indicates insufficient testing of high-load scenarios.

**Testing gaps**

- Insufficient load testing
- No stress testing
- Limited concurrency testing
- Inadequate performance test data
- Lack of realistic user simulation

**Test planning**

Performance requirements should be identified during planning.

Test cases should represent expected peak users.

For example:

- Normal load
- Expected peak load
- Beyond-peak load
- Long-duration usage

**Example**

If 5,000 students access an online learning system simultaneously, the system should be tested with approximately that workload before deployment.

**Improvements**

- Load testing
- Stress testing
- Performance monitoring
- Capacity planning
- Automated performance testing

**14. Early Involvement of Testers in SDLC**

Testers should be involved from the **requirements and design stages**, rather than waiting until development is completed.

**Benefits**

1. Testers can identify ambiguous requirements.
2. Missing requirements can be detected early.
3. Testability can be considered during design.
4. Test cases can be prepared early.
5. Defects are cheaper to fix early.

**Example**

If a requirement says, "The password should be secure," the tester can ask for specific rules such as minimum length, special characters and lockout attempts.

This prevents ambiguity from reaching implementation.

**Team collaboration**

- Requirement reviews
- Design reviews
- Developer–tester discussions
- Continuous testing
- Shared defect tracking

**Conclusion:** Early tester involvement improves quality, reduces rework and supports defect prevention.

CSA37-ST_CO1_AT1_Descriptive Wr…

**15. Incomplete Test Coverage**

A system may work for normal inputs but fail for untested conditions because testing coverage is incomplete.

**Identify gaps**

Check whether test cases cover:

- Functional requirements
- Positive scenarios
- Negative scenarios
- Boundary conditions
- Different user roles
- Error conditions
- Integration points

**Example**

For an age field accepting 18–60:

- 17 → Invalid
- 18 → Valid
- 30 → Valid
- 60 → Valid
- 61 → Invalid

Testing only age 30 does not provide adequate coverage.

**Techniques**

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Requirement Traceability Matrix
- Risk-based testing

**Conclusion:** Proper coverage ensures that normal, boundary and exceptional conditions are validated, increasing system reliability. CSA37-ST_CO1_AT1_Descriptive Wr…

**16. Improving Defect Reporting**

Poor defect reporting delays defect identification and resolution.

A good defect report should contain:

1. Defect ID
2. Clear title
3. Module
4. Environment
5. Preconditions
6. Steps to reproduce
7. Expected result
8. Actual result
9. Severity
10. Priority
11. Screenshots/logs
12. Tester details

**Example**

**Poor:**

"Login not working."

**Better:**

"Login fails when a valid username and password are entered after session timeout. Expected: user should be redirected to dashboard. Actual: application displays 500 error."

Structured reporting allows developers to understand and reproduce the defect quickly.

**Conclusion:** Standardized defect reports improve communication, tracking and resolution speed. CSA37-ST_CO1_AT1_Descriptive Wr…

**17. Process-Based Testing Across Modules**

Testing consistency can be improved by establishing a **standard testing process** for all modules.

**Standard process**

**Requirement Analysis → Test Planning → Test Design → Test Execution → Defect Reporting → Retesting → Regression Testing → Test Closure**

Every module should follow the same basic procedure.

**Example**

For an e-commerce application:

- Login module
- Product module
- Cart module
- Payment module

Each module uses standardized test-case templates, defect severity definitions and reporting formats.

**Methods to ensure consistency**

- Standard test-case templates
- Common test standards
- Test reviews
- Checklists
- Shared defect repository
- Automation
- Regular test audits

**Conclusion:** Process-based testing reduces variation between teams and ensures consistent quality across modules. CSA37-ST_CO1_AT1_Descriptive Wr…

**18. Boundary Testing for Edge Cases**

Edge cases are often missed because testers mainly test normal inputs.

**Boundary Value Analysis** focuses on values at and around boundaries.

**Example**

Suppose a system accepts withdrawal amounts from ₹100 to ₹10,000.

Test:

- ₹99 → Invalid
- ₹100 → Valid
- ₹101 → Valid
- ₹9,999 → Valid
- ₹10,000 → Valid
- ₹10,001 → Invalid

This can expose defects in conditions such as <, <=, > and >=.

**Improvements**

- Identify boundaries from requirements.
- Test minimum and maximum values.
- Test just below and above boundaries.
- Include negative and zero values where applicable.
- Add boundary cases to regression suites.

**Conclusion:** Boundary testing is highly effective for finding defects that occur at limits and prevents failures after deployment. CSA37-ST_CO1_AT1_Descriptive Wr…

**19. Defect Documentation and Repository**

A defect repository should maintain complete and organized information about every defect.

**Important fields**

- Defect ID
- Description
- Date
- Module
- Tester
- Severity
- Priority
- Steps to reproduce
- Expected result
- Actual result

- Status
- Root cause
- Resolution
- Release

**Benefits**

1. Easy defect tracking.
2. Avoids duplicate defect reports.
3. Provides historical information.
4. Helps identify defect trends.
5. Supports regression test design.
6. Helps perform root-cause analysis.

**Example:** If several defects are repeatedly reported in the payment module, repository analysis can identify payment validation as a high-risk area.

**Conclusion:** Proper defect documentation improves traceability, communication and long-term defect prevention. CSA37-ST_CO1_AT1_Descriptive Wr…

**20. Redesigning Testing for Banking Transaction Processing**

Repeated transaction failures indicate that testing is not sufficiently systematic.

**Redesigned approach**

**1. Requirement analysis**

Identify transaction rules, limits, account validation and security requirements.

**2. Test planning**

Define scope, resources, test environment, risks and coverage.

**3. Test case design**

Use:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Negative Testing

**4. Functional testing**

Verify successful and failed transactions.

**5. Integration testing**

Check communication between application, database, payment gateway and banking services.

**6. Performance testing**

Test large numbers of simultaneous transactions.

**7. Security testing**

Verify authentication, authorization and transaction protection.

**8. Regression testing**

Every fix should be followed by regression testing.

**Example**

For a ₹5,000 transfer, verify:

- Correct debit from sender
- Correct credit to receiver
- Transaction status
- Updated balances
- Transaction history
- Receipt generation

Also test insufficient balance, invalid account, network failure and duplicate transaction.

**Conclusion**

Structured test planning, systematic test design, automation, defect tracking, regression testing and risk-based testing can significantly improve defect detection and transaction reliability.

***Code of Conduct:***

*I **Haritha D / 192421282** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: Haritha***

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Concepts | 25% | Demonstrates thorough, accurate understanding of concepts | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor understanding; major misconceptions |
| Explanation | 25% | Detailed, well-explained answers with relevant examples | Clear explanation with adequate details | Limited explanation; lacks depth | Poor or unclear explanation |
| Application | 20% | Effectively applies concepts with strong analytical ability | Applies concepts with minor errors | Limited application with weak analysis | No application or incorrect analysis |
| Organization | 15% | Well-structured answers with logical flow and coherence | Organized with minor issues in flow | Basic structure, lacks clarity | Poor organization, incoherent answers |
| Accuracy | 10% | Completely accurate and covers all required points | Mostly accurate with minor omissions | Partially correct with missing points | Incorrect answers with major gaps |
| Clarity | 5% | Clear, neat, professional presentation | Understandable with minor issues | Limited clarity, readability issues | Poorly written, difficult to understand |